

СОВРЕМЕННЫЕ ТЕХНОЛОГИИ РАЗРАБОТКИ WEB-ПРИЛОЖЕНИЙ

Лекция 11b

ООП в JavaScript

- 1. ООП в функциональном стиле**
- 2. ПРОТОТИП ОБЪЕКТА**
- 3. ПРИМИТИВЫ**
- 4. НАСЛЕДОВАНИЕ КЛАССОВ**
- 5. ПРИМЕСИ**
- 6. КЛАССЫ**

ООП в ФУНКЦИОНАЛЬНОМ СТИЛЕ

Классом называют шаблон/программный код, предназначенный для создания объектов и методов.
класс User написан в «функциональном» стиле.

```
function User(name) {  
  this.sayHi = function() { alert( "Привет, я " + name ); }; }  
  
var vasya = new User("Вася"); // создали пользователя  
vasya.sayHi(); // пользователь умеет говорить "Привет"
```

- . *Внутренний интерфейс* – это свойства и методы, доступ к которым может быть осуществлен только из других методов объекта, их также называют «приватными» (есть и другие термины, встретим их далее).
- . *Внешний интерфейс* – это свойства и методы, доступные снаружи объекта, их называют «публичными».

```
function CoffeeMachine(power) {  
    this.waterAmount = 0; // количество воды в кофеварке  
    alert( 'Создана кофеварка мощностью: ' + power + ' ватт' );  
}
```

// создать кофеварку

```
var coffeeMachine = new CoffeeMachine(100);
```

// залить воды

```
coffeeMachine.waterAmount = 200;
```

Локальные переменные, включая параметры конструктора, можно считать приватными свойствами.

Свойства, записанные в this, можно считать публичными.

публичный метод run, запускающий кофеварку, вспомогательные внутренние методы getBoilTime и onReady:

```
function CoffeeMachine(power) { this.waterAmount = 0;
// расчёт времени для кипячения
function getBoilTime() { return 1000; // здесь будет формула
} // что делать по окончании процесса
function onReady() { alert( 'Кофе готов!' ); }
this.run = function() {
// setTimeout - запустит onReady через getBoilTime() миллисекунд
setTimeout(onReady, getBoilTime()); };}
var coffeeMachine = new CoffeeMachine(100);
coffeeMachine.waterAmount = 200; coffeeMachine.run();
```

самый удобный и часто применяемый путь указания контекста состоит в том, чтобы предварительно скопировать `this` во вспомогательную переменную и обращаться из внутренних функций уже к ней:

```
function CoffeeMachine(power) { this.waterAmount = 0;
    var WATER_HEAT_CAPACITY = 4200;
    var self = this;
    function getBoilTime() {
        return self.waterAmount * WATER_HEAT_CAPACITY * 80 / power;    }
    function onReady() {    alert( 'Кофе готов!' );    }
    this.run = function() {    setTimeout(onReady, getBoilTime());    }; }
var coffeeMachine = new CoffeeMachine(100000);
coffeeMachine.waterAmount = 200; coffeeMachine.run();
```

Теперь `getBoilTime` получает `self` из замыкания.

- чтобы это работало, мы не должны изменять `self`, а все приватные методы, которые хотят иметь доступ к текущему объекту, должны использовать внутри себя `self` вместо `this`.
- Вместо `self` можно использовать любое другое имя переменной, например `var me = this`.

ГЕТТЕРЫ И СЕТТЕРЫ

Для лучшего контроля над свойством его делают приватным, а запись значения осуществляется через специальный метод, который называют «*сеттер*» (setter method).

Для того, чтобы дать возможность внешнему коду узнать его значение, создают специальную функцию – «*геттер*» (getter method).

```
function CoffeeMachine(power, capacity) { //...
  this.setWaterAmount = function(amount) {
    if (amount < 0) {
      throw new Error("Значение должно быть положительным");
    }
    if (amount > capacity) {
      throw new Error("Нельзя залить воды больше, чем " + capacity);
    }
    waterAmount = amount;
  };
  this.getWaterAmount = function() { return waterAmount; };
  var coffeeMachine = new CoffeeMachine(1000, 500);
  coffeeMachine.setWaterAmount(450);
  alert( coffeeMachine.getWaterAmount() ); // 450
```

```
function CoffeeMachine(power, capacity) {  
    var waterAmount = 0; this.waterAmount = function(amount) {  
        // вызов без параметра, режим геттера, возвращаем свойство  
        if (!arguments.length) return waterAmount;  
        // иначе режим сеттера  
        if (amount < 0) {  
            throw new Error("Значение должно быть положительным");  
        } if  
(amount > capacity) {  
            throw new Error("Нельзя залить воды больше, чем " + capacity);  
        }  
        waterAmount = amount; };  
}  
var coffeeMachine = new CoffeeMachine(1000, 500);  
coffeeMachine.waterAmount(450);  
alert( coffeeMachine.waterAmount() ); // 450
```

ФУНКЦИОНАЛЬНОЕ НАСЛЕДОВАНИЕ

Базовый класс «машина» Machine будет реализовывать общего вида методы «включить» enable() и «выключить» disable():

```
function Machine() {  
  var enabled = false;  
  this.enable = function() {  
    enabled = true; };  
  this.disable = function() {  
    enabled = false; };  
}
```

```
function CoffeeMachine(power) {  
  Machine.call(this); // наследовать  
  var waterAmount = 0;  
  this.setWaterAmount = function(amount) {  
    waterAmount = amount; };  
}
```

```
var coffeeMachine = new CoffeeMachine(10000);  
coffeeMachine.enable();  
coffeeMachine.setWaterAmount(100);  
coffeeMachine.disable();
```

Наследование реализовано вызовом `Machine.call(this)` в начале конструктора `CoffeeMachine`.

- Он вызывает функцию `Machine`, передавая ей в качестве контекста `this` текущий объект. `Machine`, в процессе выполнения, записывает в `this` различные полезные свойства и методы, в нашем случае `this.enable` и `this.disable`.
- Далее конструктор `CoffeeMachine` продолжает выполнение и может добавить свои свойства и методы.

ЗАЩИЩЁННЫЕ СВОЙСТВА

Чтобы наследник имел доступ к свойству, оно должно быть записано в `this`.

При этом, чтобы обозначить, что свойство является внутренним, его имя начинают с подчёркивания `_`.

```
function Machine() {  
    this._enabled = false; // вместо var enabled  
    this.enable = function() {  
        this._enabled = true; };  
    this.disable = function() {  
        this._enabled = false; };}  
function CoffeeMachine(power) {  
    Machine.call(this);  
    this.enable();  
    alert( this._enabled ); // true  
}  
var coffeeMachine = new CoffeeMachine(10000);
```


Для наследования конструктор потомка вызывает родителя в своём контексте через `apply`. После чего может добавить свои переменные и методы:

```
function CoffeeMachine(params) {  
  // универсальный вызов с передачей любых аргументов  
  Machine.apply(this, arguments);  
  this.coffeePublicProperty = ...}
```

```
var coffeeMachine = new CoffeeMachine(...);  
coffeeMachine.publicProperty();  
coffeeMachine.coffeePublicProperty();
```

ПРОТОТИП ОБЪЕКТА

Прототипы - это механизм, с помощью которого объекты JavaScript наследуют свойства друг от друга.

Объекты в JavaScript можно организовать в цепочки так, чтобы свойство, не найденное в одном объекте, автоматически искалось бы в другом.

Связующим звеном выступает специальное свойство `__proto__`.

Если один объект имеет специальную ссылку `__proto__` на другой объект, то при чтении свойства из него, если свойство отсутствует в самом объекте, оно ищется в объекте `__proto__`.

В спецификации ECMAScript – свойство `__proto__` обозначено как `[[Prototype]]`.

```
var animal = {  
  eats: true};  
var rabbit = { jumps: true};  
  
rabbit.__proto__ = animal;  
// в rabbit можно найти оба свойства  
alert( rabbit.jumps ); // true  
alert( rabbit.eats ); // true
```

1. Первый alert выводит свойство jumps объекта rabbit.
2. Второй alert хочет вывести rabbit.eats, ищет его в самом объекте rabbit, не находит – и продолжает поиск в объекте rabbit.__proto__, то есть, в данном случае, в animal.

animal

eats: true

__proto__

rabbit

jumps: true



Объект, на который указывает ссылка `__proto__`, называется *«прототипом»*.

В данном случае `animal` является прототипом для `rabbit`.

Также говорят, что объект `rabbit` *«прототипно наследует»* от `animal`.

Обратим внимание – прототип используется исключительно при чтении. Запись значения, например, `rabbit.eats = value` или удаление `delete rabbit.eats` – работает напрямую с объектом.

```
var animal = { eats: true};  
var rabbit = { jumps: true, eats: false};  
rabbit.__proto__ = animal;  
alert( rabbit.eats ); // false, свойство взято из rabbit
```

Другими словами, прототип – это «резервное хранилище свойств и методов» объекта, автоматически используемое при поиске.

У объекта, который является __proto__, может быть свой __proto__, у того – свой, и так далее. При этом свойства будут искаться по цепочке.

Обычный цикл `for..in` не делает различия между свойствами объекта и его прототипа.

Он перебирает всё, например:

```
var animal = { eats: true};  
var rabbit = { jumps: true, __proto__: animal};
```

```
for (var key in rabbit) {  
    console.log ( key + " = " + rabbit[key] );  
// выводит и "eats" и "jumps"}
```

```
"jumps = true"
```

```
"eats = true"
```

Чтобы посмотреть, что находится именно в самом объекте, а не в прототипе используют

Вызов `obj.hasOwnProperty(prop)` возвращает `true`, если свойство `prop` принадлежит самому объекту `obj`, иначе `false`.

чтобы перебрать свойства самого объекта, достаточно профильтровать key через hasOwnProperty:

```
var animal = { eats: true};  
var rabbit = { jumps: true, __proto__: animal};  
for (var key in rabbit) {  
    if (!rabbit.hasOwnProperty(key)) continue;  
    // пропустить "не свои" свойства  
    alert( key + " = " + rabbit[key] ); }  
// выводит только "jumps = true"
```


Зачастую объекты используют для хранения произвольных значений по ключу, как коллекцию:

```
var data = {};
```

```
data.text = "Привет";
```

```
data.age = 35;
```

// ... При дальнейшем поиске в этой коллекции мы найдём не только text и age, но и встроенные функции:

```
var data = {};
```

```
alert(data.toString);
```

// функция, хотя мы её туда не записывали Это может быть неприятным сюрпризом и приводить к ошибкам, если названия свойств приходят от посетителя и могут быть произвольными.

```
function toString() { [native code] }
```

можем исключать свойства, не принадлежащие самому объекту:

```
var data = {};
```

```
// выведет toString только если оно записано в сам объект
```

```
alert(data.hasOwnProperty('toString') ? data.toString : undefined);
```

проще:

```
var data = Object.create(null);  
data.text = "Привет";  
alert(data.text); // Привет  
alert(data.toString); // undefined
```

Объект, создаваемый при помощи `Object.create(null)` не имеет прототипа, а значит в нём нет лишних свойств.

методы для работы с `__proto__` (существуют по историческим причинам):

Чтение: `Object.getPrototypeOf(obj)`

Возвращает `obj.__proto__` (кроме IE8-)

Запись: `Object.setPrototypeOf(obj, proto)`

Устанавливает `obj.__proto__ = proto` (кроме IE10-).

Создание объекта с прототипом: `Object.create(proto, descriptors)`

Создаёт пустой объект с `__proto__`, равным первому аргументу (кроме IE8-), второй необязательный аргумент может содержать дескрипторы свойств.

```
var animal = { eats: true};
```

```
function Rabbit(name) {
```

```
  this.name = name;
```

```
  this.__proto__ = animal;}
```

```
var rabbit = new Rabbit("Кроль");
```

//у всех объектов, которые создаются new Rabbit, прототип animal

```
alert( rabbit.eats ); // true, из прототипа
```

СВОЙСТВО F.PROTOTYPE

кросс-браузерный способ.

Чтобы новым объектам автоматически устанавливать прототип, для конструктора устанавливается свойство prototype.

При создании объекта через new, в его прототип __proto__ записывается ссылка из prototype функции-конструктора.

```
var animal = { eats: true};
```

```
function Rabbit(name) {  
  this.name = name;}  
Rabbit.prototype = animal;
```

```
var rabbit = new Rabbit("White Rabbit"); // rabbit.__proto__ == animal  
alert( rabbit.eats ); // true
```

`Rabbit.prototype = animal` : *"При создании объекта через `new Rabbit` запишется `__proto__ = animal`".*

Свойство `prototype`

- имеет смысл только у конструктора
- можно указать на любом объекте, но особый смысл оно имеет, лишь если назначено функции-конструктору.
- без вызова оператора `new`, оно вообще ничего не делает, его единственное назначение – указывать `__proto__` для новых объектов.
- Значением `prototype` может быть только объект
- в это свойство можно записать что угодно.
- при работе `new` будет использовано лишь в том случае, если это объект. Примитивное значение, такое как число или строка, будет проигнорировано.

СВОЙСТВО CONSTRUCTOR

У каждой функции по умолчанию уже есть свойство `prototype`.

Оно содержит объект такого вида:

```
function Rabbit() {}
```

//генерируется автоматически.

```
Rabbit.prototype = { constructor: Rabbit};
```


Проверка:

```
function Rabbit() {}
```

```
// в Rabbit.prototype есть свойство: constructor
```

```
alert( Object.getOwnPropertyNames(Rabbit.prototype) );
```

```
// constructor
```

```
// оно равно Rabbit
```

```
alert( Rabbit.prototype.constructor == Rabbit ); // true
```

Можно его использовать для создания объекта с тем же конструктором, что и данный:

```
function Rabbit(name) {
```

```
  this.name = name;
```

```
  alert( name );}
```

```
var rabbit = new Rabbit("Кроль");
```

```
var rabbit2 = new rabbit.constructor("Крольчиха");
```

Эта возможность бывает полезна, когда, получив объект, мы не знаем в точности, какой у него был конструктор (например, сделан вне нашего кода), а нужно создать такой же.

при перезаписи

```
Rabbit.prototype = { jumps: true }
```

свойства `constructor` больше не будет, его легко потерять

если мы хотим, чтобы была возможность получить конструктор, то можно при перезаписи гарантировать наличие `constructor` вручную:

```
Rabbit.prototype = { jumps: true, constructor: Rabbit};
```

Либо можно поступить аккуратно и добавить свойства к
встроенному prototype без его замены:

// сохранится встроенный constructor

```
Rabbit.prototype.jumps = true
```

В JavaScript есть встроенные объекты: Date, Array, Object и другие. Они используют прототипы и демонстрируют организацию «псевдоклассов» на JavaScript, которую можно применить

создадим пустой объект и выведем его.

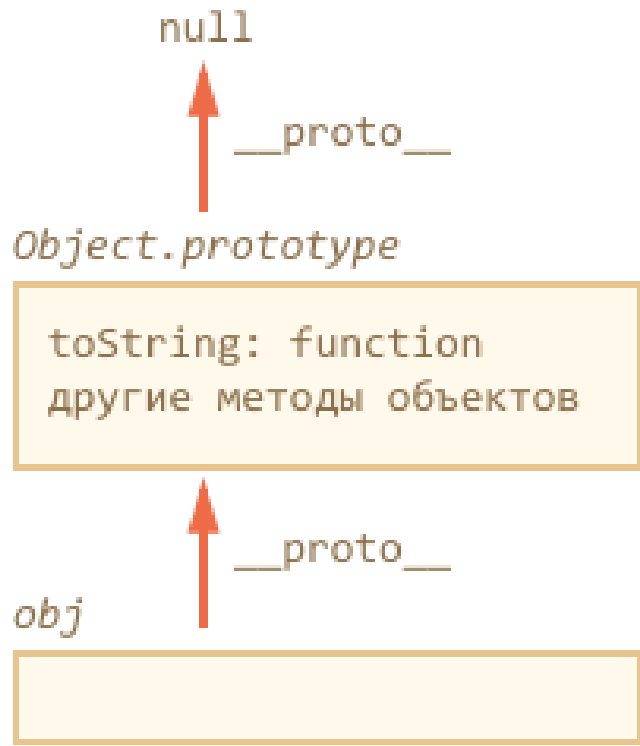
```
var obj = {};
```

```
alert( obj ); // "[object Object]" ?
```

это сделал метод `toString`, который находится в прототипе `obj.__proto__`:

```
alert( {}.__proto__.toString ); // function toString
```

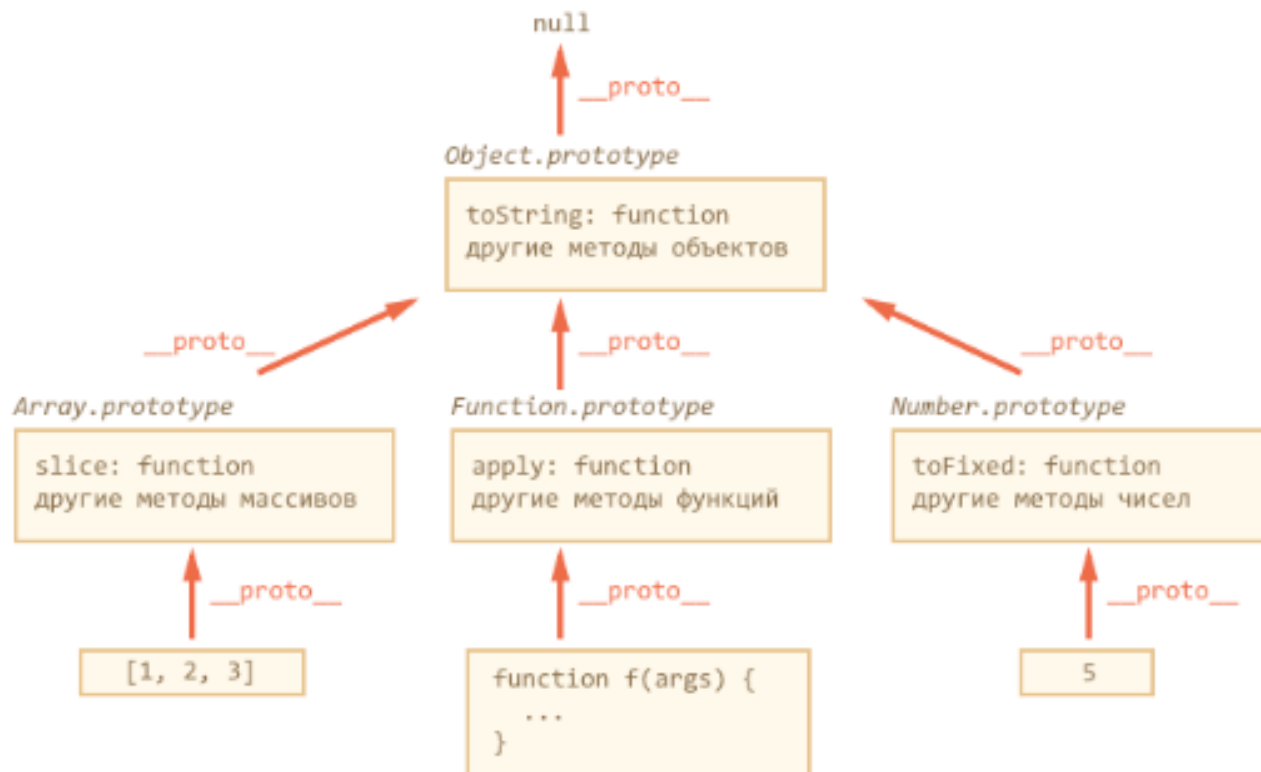
1. Запись `obj = {}` является краткой формой `obj = new Object`, где `Object` – встроенная функция-конструктор для объектов.
2. При выполнении `new Object`, создаваемому объекту ставится `__proto__` по `prototype` конструктора, который в данном случае равен встроенному `Object.prototype`.
3. В дальнейшем при обращении к `obj.toString()` – функция будет взята из `Object.prototype`.



```
var obj = {};  
// метод берётся из прототипа?  
alert( obj.toString == Object.prototype.toString );  
// true, да  
// проверим, правда ли что __proto__ это  
// Object.prototype?  
alert( obj.__proto__ == Object.prototype );  
// true  
// А есть ли __proto__ у Object.prototype?  
  
alert( obj.__proto__.__proto__ );  
// null, нет
```

ВСТРОЕННЫЕ «КЛАССЫ» В JAVASCRIPT

Такой же подход используется в массивах Array, функциях Function и других объектах. Встроенные методы для них находятся в Array.prototype, Function.prototype и т.п.



Поэтому говорят, что "все объекты наследуют от Object", а если более точно, то от Object.prototype.

«Псевдоклассом» или, более коротко, «классом», называют функцию-конструктор вместе с её prototype. Такой способ объявления классов называют «прототипным стилем ООП».

При наследовании часть методов переопределяется, например, у массива Array есть свой toString, который выводит элементы массива через запятую:

```
var arr = [1, 2, 3];  
alert( arr ); // 1,2,3 <-- результат Array.prototype.toString
```

Object.prototype

```
toString: function  
...
```



Array.prototype

```
toString: function  
...
```



```
[1, 2, 3]
```

у `Object.prototype` есть свой `toString`, но так как в `Array.prototype` он ищется первым, то берётся именно вариант для массивов

ПРИМИТИВЫ

Примитивы не являются объектами, но методы берут из соответствующих прототипов: `Number.prototype`, `Boolean.prototype`, `String.prototype`.

По стандарту, если обратиться к свойству числа, строки или логического значения, то будет создан объект соответствующего типа, например `new String` для строки, `new Number` для чисел, `new Boolean` – для логических выражений.

Далее будет произведена операция со свойством или вызов метода по обычным правилам, с поиском в прототипе, а затем этот объект будет уничтожен.

```
var user = "Вася"; // создали строку (примитив)  
alert( user.toUpperCase() ); // ВАСЯ  
// был создан временный объект new String  
// вызван метод  
// new String уничтожен, результат возвращён
```

// попытаемся записать свойство в строку:

```
var user = "Вася";
```

```
user.age = 30;
```

```
alert( user.age ); // undefined
```

Свойство age было записано во временный объект, который был тут же уничтожен, так что смысла в такой записи немного.

СВОИ КЛАССЫ НА ПРОТОТИПАХ

Чтобы объявить свой класс, нужно:

1. Объявить функцию-конструктор.
2. Записать методы и свойства, нужные всем объектам класса, в `prototype`.

Опишем класс Animal:

// конструктор

```
function Animal(name) { this.name = name; this.speed = 0;}
```

// методы в прототипе

```
Animal.prototype.run = function(speed) {
```

```
  this.speed += speed; alert( this.name + ' бежит, скорость ' + this.speed );  
};
```

```
Animal.prototype.stop = function() {
```

```
  this.speed = 0; alert( this.name + ' стоит' );  };
```

```
var animal = new Animal('Зверь');
```

```
alert( animal.speed ); // 0, свойство взято из прототипа
```

```
animal.run(5); // Зверь бежит, скорость 5
```

```
animal.run(5); // Зверь бежит, скорость 10
```

```
animal.stop(); // Зверь стоит
```

В объекте `animal` будут храниться свойства конкретного экземпляра: `name` и `speed`, а общие методы – в прототипе.

Достоинства

Функциональный стиль записывает в каждый объект и свойства и методы, а прототипный – только свойства. Поэтому прототипный стиль – быстрее и экономнее по памяти.

Недостатки

При создании методов через прототип, мы теряем возможность использовать локальные переменные как приватные свойства, у них больше нет общей области видимости с конструктором.

НАСЛЕДОВАНИЕ КЛАССОВ В JAVASCRIPT

то есть когда объекты, создаваемые, к примеру, через `new Admin`, должны иметь все методы, которые есть у объектов, создаваемых через `new User`, и ещё какие-то свои.

Внутри JavaScript на примере `Array`, который наследует от `Object`:

Методы массивов `Array` хранятся в `Array.prototype`.

`Array.prototype` имеет прототипом `Object.prototype`.

Поэтому когда экземпляры класса `Array` хотят получить метод массива – они берут его из своего прототипа, например `Array.prototype.slice`.

Если же нужен метод объекта, например, `hasOwnProperty`, то его в `Array.prototype` нет, и он берётся из `Object.prototype`.

запустить в консоли команду `console.dir([1,2,3])`.
Вывод в Chrome будет примерно таким:

```
> console.dir([1,2,3])
▼ Array[3] ⓘ
  0: 1
  1: 2
  2: 3
  length: 3
  ▼ __proto__: =Array.prototype
    ► concat: function concat() { [native code] }
    ► ...
    ► unshift: function unshift() { [native code] }
    ▼ __proto__: =Object.prototype
      ► ...
      ► constructor: function Object() { [native code] }
      ► hasOwnProperty: function hasOwnProperty() { [native code] }
      ► isPrototypeOf: function isPrototypeOf() { [native code] }
      ► ...
```

объявим класс Rabbit, который будет наследовать от Animal

Для того, чтобы наследование работало, объект rabbit = new Rabbit должен использовать свойства и методы из своего прототипа Rabbit.prototype, а если их там нет, то – свойства и метода родителя, которые хранятся в Animal.prototype.

порядок поиска свойств и методов должен быть таким: rabbit -> Rabbit.prototype -> Animal.prototype, по аналогии с тем, как это сделано для объектов и массивов.

Для этого можно поставить ссылку

__proto__ с Rabbit.prototype на Animal.prototype.

Можно сделать это так:

```
Rabbit.prototype.__proto__ = Animal.prototype;
```

// 1. Конструктор Animal

```
function Animal(name) {  
    this.name = name; this.speed = 0;  
}
```

// 1.1. Методы -- в прототип

```
Animal.prototype.stop = function() {  
    this.speed = 0; alert( this.name + ' стоит' );  
}
```

```
Animal.prototype.run = function(speed) {  
    this.speed += speed;  
    alert( this.name + ' бежит, скорость ' + this.speed );  
};
```

// 2. Конструктор Rabbit

```
function Rabbit(name) {  
  this.name = name; this.speed = 0;  
}
```

// 2.1. Наследование

```
Rabbit.prototype = Object.create(Animal.prototype);  
Rabbit.prototype.constructor = Rabbit;
```

// 2.2. Методы

```
Rabbit.prototype.jump = function() {  
  this.speed++;  
  alert( this.name + ' прыгает, скорость ' + this.speed );  
}
```

Animal.prototype

```
run: function  
stop: function
```



__proto__

Rabbit.prototype

```
jump: function
```



__proto__

rabbit

```
name: "Кроль"  
speed: 0
```

В prototype по умолчанию всегда находится свойство constructor, указывающее на функцию-конструктор. В частности, `Rabbit.prototype.constructor == Rabbit`. Если мы рассчитываем использовать это свойство, то при замене prototype через `Object.create` нужно его явно сохранить:

```
Rabbit.prototype = Object.create(Animal.prototype);  
Rabbit.prototype.constructor = Rabbit;
```

ВЫЗОВ КОНСТРУКТОРА РОДИТЕЛЯ

Чтобы упростить поддержку кода, имеет смысл не дублировать код конструктора `Animal`, а напрямую вызвать его:

```
function Rabbit(name) { Animal.apply(this, arguments);}
```

Такой вызов запустит функцию `Animal` в контексте текущего объекта, со всеми аргументами, она выполнится и запишет в `this` всё, что нужно.

ПЕРЕОПРЕДЕЛЕНИЕ МЕТОДА

переопределим метод run():

```
Rabbit.prototype.run = function(speed) {  
  this.speed++; this.jump();  
};
```

Вызов rabbit.run() теперь будет брать run из своего прототипа

Animal.prototype

```
run: function  
stop: function
```



__proto__

Rabbit.prototype

```
jump: function  
run: function
```



__proto__

rabbit

```
name: "Кроль"  
speed: 0
```


ВЫЗОВ МЕТОДА РОДИТЕЛЯ ВНУТРИ СВОЕГО

Более частая ситуация – когда мы хотим не просто заменить метод на свой, а взять метод родителя и расширить его.

```
Rabbit.prototype.run = function() {  
    // вызвать метод родителя, передав ему текущие аргументы  
    Animal.prototype.run.apply(this, arguments);  
    this.jump();  
}
```

Если вызвать просто `Animal.prototype.run()`, то в качестве `this` функция `run` получит `Animal.prototype`, а это неверно, нужен текущий объект.

```
var rabbit = new Rabbit('Кроль');  
rabbit.run();
```

Такое наследование лучше функционального стиля, так как не дублирует методы в каждом объекте.

Если конструктор родителя имеет какое-то поведение, которое нужно переопределить в потомке, то в функциональном стиле это невозможно.

ПРОВЕРКА КЛАССА: "INSTANCEOF"

Вызов `obj instanceof Constructor` возвращает `true`, если объект принадлежит классу `Constructor` или классу, наследующему от него.

```
function Rabbit() {}
```

```
// создаём объект
```

```
var rabbit = new Rabbit();
```

```
// проверяем -- этот объект создан Rabbit?
```

```
alert( rabbit instanceof Rabbit ); // true, верно
```

Массив `arr` принадлежит классу `Array`, но также и является объектом `Object`. Это верно, так как массивы наследуют от объектов:

```
var arr = [];
```

```
alert( arr instanceof Array ); // true
```

```
alert( arr instanceof Object ); // true
```

Алгоритм проверки `obj instanceof Constructor`:

1. Получить `obj.__proto__`
2. Сравнить `obj.__proto__` с `Constructor.prototype`
3. Если не совпадает, тогда заменить `obj` на `obj.__proto__` и повторить проверку на шаге 2 до тех пор, пока либо не найдется совпадение (результат `true`), либо цепочка прототипов не закончится (результат `false`).

- сама функция-конструктор не участвует в процессе проверки!
Важна только цепочка прототипов для проверяемого объекта.
- Это может приводить к забавному результату и даже ошибкам в проверке при изменении prototype

ПРИМЕСИ

- Примесь (англ. `mixin`) – класс или объект, реализующий какое-либо чётко выделенное поведение. Используется для уточнения поведения других классов, не предназначен для самостоятельного использования.

Самый простой вариант примеси – это объект с полезными методами, которые мы просто копируем в нужный прототип.

```
// примесь
```

```
var sayHiMixin = {  
  sayHi: function() {  
    alert("Привет " + this.name); },  
  sayBye: function() {  
    alert("Пока " + this.name); }  
};
```

```
// использование:
```

```
function User(name) { this.name = name;}
```

```
// передать методы примеси
```

```
for(var key in sayHiMixin)
```

```
User.prototype[key] = sayHiMixin[key]; // User "умеет" sayHi
```

```
new User("Вася").sayHi(); // Привет Вася
```

Объекты в JavaScript

1. Любой объект – это значение типа данных `object`.
2. Объект – это неупорядоченная *коллекция свойств*.
3. Каждое свойство имеет *имя* и *значение*.
4. Имена свойств – это *строки*, значения свойств могут иметь *любой тип* (число, строка, объект, функция, ...).
5. Объект позволяет *читать и записывать* значения своих свойств.

6. Объекты являются *динамическими* – обычно они позволяют добавлять и удалять свои свойства.
7. Используется *прототипная объектная модель*: любой объект может быть связан с другим объектом (*прототипом*) и иметь доступ к свойствам прототипа.
8. *Классом* называется набор объектов с общим прототипом.
9. Элементы классического ООП моделируются.

10. Объекты используют *ссылочную* семантику при присваивании.

11. Переменной с объектом можно присвоить значение `null`. Это приводит к потере ссылки на объект.

12. Движки реализуют автоматическую сборку мусора.

```
class Person {  
  constructor(firstName, lastName) {  
    this.firstName = firstName  
    this.lastName = lastName  
  }  
  getFullName() {  
    return this.firstName + ' ' + this.lastName  
  }  
}
```

ключевое слово class из ES6

```
let person = new Person('Dan', 'Abramov')
person.getFullName() //> "Dan Abramov"
// Мы можем использовать акцессор или получить доступ напрямую
person.firstName //> "Dan"
person.lastName //> "Abramov"
```

```
class User extends Person {  
    constructor(firstName, lastName, email, password) {  
        super(firstName, lastName)  
        this.email = email  
        this.password = password  
    }  
    getEmail() {  
        return this.email  
    }  
    getPassword() {  
        return this.password  
    }  
}
```

```
function App() {  
  let user = new User('Dan',  
                        'Abramov',  
                        'dan@abramov.com',  
                        'iLuvES6')  
  
  user.getFullName() //> "Dan Abramov"  
  user.getEmail() //> "dan@abramov.com"  
  user.getPassword() //> "iLuvES6"  
  user.firstName //> "Dan"  
  user.lastName //> "Abramov"  
  user.email //> "dan@abramov.com"  
  user.password //> "iLuvES6"  
}
```

Пример: класс и работа с ним

```
class Point {  
  constructor(x, y) {  
    this.x = x;  
    this.y = y;  
  }  
  toString() {  
    return '(' + this.x + ', ' + this.y + ')';  
  }  
}
```

```
let p = new Point(3, 5);  
alert(p.toString());
```

Функция constructor запускается при создании new User, остальные методы записываются в User.prototype

Во что (примерно) превращается class

```
function Point(x, y) {  
    // проверка того, что вызов Point был с new  
    // иначе генерируется исключение TypeError  
    this.x = x;  
    this.y = y;  
}
```

```
Point.prototype.toString = function () {  
    return '(' + this.x + ', ' + this.y + ')';  
};
```


Особенности конструкции class

- Конструктор нельзя вызвать без `new`, иначе исключение `TypeError`.
- Объявление класса ведёт себя как `let`. Оно видно только в текущем блоке и только в коде, который находится ниже объявления.
- Методы внутри `class` являются методами объекта (см. *методы объекта в литерале объекта*).
- Все методы работают в строгом режиме.
- Все методы класса не перечислимы в цикле `for/in`.

Классы в ECMAScript 2015 – нюансы

Классы не могут содержать обычных свойств. Но в классе можно описать *методы* чтения и записи свойств.

Также для имён методов разрешено использовать выражения в квадратных скобках (как в литералах объектов).

Классы в ECMAScript 2015 – нюансы

```
class User {  
    constructor(firstName, lastName) {  
        this.firstName = firstName;  
        this.lastName = lastName;  
    }  
    get fullName() { return `${this.firstName}  
${this.lastName}`; }  
  
    ["test".toUpperCase]() { alert("PASSED!"); }  
};  
let u = new User('Mary', 'Mor');  
alert(u.fullName);  
u.TEST();
```

Конструкция class как выражение

Конструкция `class` может использоваться как выражение. Возможное применение – передавать такое выражение в *фабрики объектов*.

```
let Point = class {  
  constructor(x, y) {  
    this.x = x;  
    this.y = y;  
  }  
  
  toString() { return `${this.x}, ${this.y}`; }  
}
```

Статические элементы класса

Статические элементы класса объявляются с модификатором `static`.

По сути, речь идёт о добавлении свойств к объекту, описывающему функцию-конструктор.

Статические элементы – пример

```
class Point {  
    constructor(x, y) {  
        this.x = x;  
        this.y = y;  
    }  
    static createPoint(x) { return new Point(x, 0); }  
    static get Zero() { return new Point(0, 0); }  
    toString() { return `${this.x}, ${this.y}`; }  
}  
// работа со статическими элементами  
alert(Point.createPoint(1).toString());  
alert(Point.Zero.toString());
```

Наследование классов

Конструкция `class` позволяет при описании класса указать **базовый класс** при помощи ключевого слова `extends`. Как и в классическом ООП, **наследник** получает доступ к методам базового класса.

В конструкторе и методах наследника доступ к предку можно получить при помощи ключевого слова `super`.

Внимание: в конструкторе наследника **должен** быть вызов конструктора предка через `super()`, и этот вызов должен предварять обращение к `this`.

Наследование классов – пример

```
// это будет базовый класс
class Point {
    constructor(x, y) {
        this.x = x;
        this.y = y;
    }
    toString() {
        return `${this.x}, ${this.y}`;
    }
}
```


Наследование классов – пример

```
// класс-наследник
class ColorPoint extends Point {
    constructor(x, y, color) {
        super(x, y);
        this.color = color;
    }
    toString() {
        return super.toString()+' in '+this.color;
    }
}
```

Ключевое слово `super` в литералах

Конструкция `super` работает не только в классах, но и в литералах объектов (но только для *методов объектов*).

```
let animal = {  
  walk() { alert("I'm walking"); }  
};  
let rabbit = {  
  __proto__: animal,  
  walk() { super.walk(); }  
};  
rabbit.walk();
```

Ключевое слово `super` в литералах

Как и почему работает `super` в методах объектов?

Дело в том, что метод объект (и только он) имеет внутренний атрибут `[[HomeObject]]` – ссылка на объект, которому метод принадлежит.

Запись `super.walk()` эквивалентна `[[HomeObject]].__proto__.walk()`

Класс без явного конструктор

В конструкции `class` можно не задавать `constructor`. В классах без предка это эквивалентно объявлению

```
constructor() {}
```

В класса-наследниках отсутствие конструктора равносильно такому объявлению:

```
constructor(...args) {  
    super(...args);  
}
```